Evaluating Prompt Strategies for Automated Pull Request Summarization Using a Large Language Model

Haritha Raj, Department of Information Science and Engineering, RV Institute of Technology and Management, Bengaluru, India

Abstract

Pull request (PR) reviewers often need to infer the purpose and consequences of a change from its code diff, particularly when the submitted description is missing or uninformative. Large language models (LLMs) can generate summaries quickly, but their usefulness may depend on how the task is phrased. This paper compares a plain instruction to summarize a PR with a structured instruction requesting three sections: Changes, Reason, and Risks. The example system is an open-source GitHub PR summarizer using webhooks, diff retrieval, and the Gemini API. The study design covered 20 merged public PRs from eight repositories. Both prompts had a 150-word limit and used a fixed Gemini model at temperature 0; two raters scored the outputs on a five-point scale in a blinded setup. The structured prompt received a mean score of 4.05, compared with 3.80 for the plain prompt. The difference was 0.25 points, with a Wilcoxon signed-rank p-value of 0.053. The structured prompt won on 11 PRs, the plain prompt on five, and four were ties. Structured summaries were also somewhat longer and slower.

Keywords: *pull requests; code review; large language models; prompt engineering; summarization; empirical study*

1. Introduction

Pull requests are a central part of collaborative software development. Before reviewing individual lines of code, a reviewer needs to understand what changed, why the change was made, and what effects or risks may follow. A clear PR description provides this context, but descriptions are sometimes absent or too vague to be useful. Liu et al. reported that more than 34% of descriptions in a dataset containing over 333,000 PRs were empty [1]. When context is missing, reviewers must reconstruct the intended change from commit messages and code diffs, which can take additional time and may contribute to shallow or delayed reviews.

LLMs make it possible to generate a summary from a code diff within seconds. A typical summarization bot receives a webhook when a PR is opened, retrieves the diff, sends it to a model, and publishes the generated text as a comment. Many design choices can affect the output, but the prompt is relatively easy to change. A simple instruction to summarize a PR

can be contrasted with a structured instruction requiring separate explanations of the changes, their rationale, and possible risks.

The structured approach may make summaries easier to scan and encourage coverage of details that a free-form response might omit. It could also introduce unnecessary text, encourage unsupported risk speculation, or force the model to divide a limited word budget among sections.

Research question: Does a structured prompt organized into Changes, Reason, and Risks produce more useful PR summaries than a plain prompt when both have a 150-word limit?

The intended comparison holds the model, input format, word limit, and PR set constant so that prompt design is the main intended difference. Human ratings are used because text-overlap metrics do not necessarily capture whether a summary helps a reviewer understand a change.

Contributions. The paper describes a controlled comparison of two prompting strategies within a working system, a blinded human-rating protocol with randomized presentation, practical considerations for selecting a prompt, and a reproducibility package containing scripts, prompts, summaries, and ratings.

2. Related Work

2.1 PR description generation

Liu et al. treated PR description generation as a summarization task and proposed a sequence-to-sequence model using commit messages and source-code comments as inputs. Their evaluation used ROUGE and human judgment [1]. Unlike a task-specific trained model, this study uses a general-purpose LLM and changes the instruction rather than developing a new model.

2.2 LLM-based code summarization

Research on LLM-based code summarization examines how well general-purpose models describe code and how prompting affects their outputs [2]. Much of this work focuses on individual functions or snippets. PR diffs are broader: they can span multiple files, combine implementation and configuration edits, include tests, and omit intent that cannot be inferred from the code alone.

2.3 Prompting strategies

Prior work shows that structured or step-oriented instructions can affect model behaviour across tasks [3]. That background motivates examining a structured output format for PR summaries, but it does not establish that the format improves reviewer usefulness in this specific setting.

2.4 Research gap

This paper focuses on a comparison of prompt formats for LLM-generated PR summaries evaluated by people. Earlier work discussed here either trains dedicated models for PR descriptions or studies code summarization at a smaller granularity.

3. System Under Study

The system is an open-source GitHub App that generates PR summaries automatically [4]. Its workflow consists of five stages:

1. GitHub sends a webhook when a PR is opened. The backend validates the request's HMAC signature.
2. The application authenticates as a GitHub App by generating a signed JSON Web Token and exchanging it for an installation access token.
3. The backend retrieves the PR diff through the GitHub API.
4. The PR title and diff are sent to the Gemini API, which generates the summary.
5. The summary is posted as a PR comment and saved in a database for display on a dashboard.

The backend uses Node.js and Express, with MongoDB for persistence. The front end is a React application that displays stored summaries. The backend and front end are hosted separately. A standalone script retrieves the title and unified diff for already-merged PRs and sends that input to the model.

4. Method

4.1 Dataset

The dataset contained 20 merged PRs from eight public repositories: three PRs each from Flask, Requests, Express, FastAPI, and Axios; two each from Click and Vite; and one from HTTPX. Selection used random seed 42. Recently closed PRs were listed for each repository, filtered to retain merged PRs, shuffled, and accepted when their diffs contained between 800 and 12,000 characters. Seven of the 20 PRs had no human-written description.

Merged PRs provide stable examples of changes accepted by maintainers, but they may be better formed than the average opened PR, potentially making them easier to summarize.

4.2 Prompts

Both prompt variants received the PR title and unified diff. The plain version asked for a summary of no more than 150 words. The structured version imposed the same total limit but required exactly three labelled sections: Changes (what was modified), Reason (why the change was made), and Risks (what reviewers should inspect or what might go wrong). The limit applied to the full structured response, not to each section.

4.3 Model settings

We used a fixed Gemini model at temperature 0, accessed through the official Python SDK. [TO DO: add the exact Gemini model name and version, and the run date.] Prompt-call order was randomized for each PR to reduce systematic effects from caching or service load.

4.4 Human evaluation

Two raters—the author and a friend who is a final-year engineering student familiar with reading code on GitHub—scored every summary. Each summary received an integer rating from 1 to 5 based on usefulness and accuracy.

Table 1. Rating rubric

Score	Interpretation
1	Misleading or unusable; incorrectly describes the PR.
2	Poor; vague or missing the main change.
3	Acceptable; correct but incomplete or generic.
4	Good; accurate, covers the main changes and some intent.
5	Excellent; accurate, concise, and clearly useful to a reviewer.

Summaries were shown under neutral labels X and Y in randomized order. The mapping between labels and prompts was kept separately until scoring was complete. Each rater scored independently, and the per-PR score was the average of the two ratings.

4.5 Measures

The measures were mean, standard deviation, and median of per-PR average ratings; prompt wins and ties; API response time; summary length and the number exceeding 150 words; and inter-rater agreement calculated using quadratic weighted Cohen's kappa.

4.6 Statistical analysis

The analysis used the paired, non-parametric Wilcoxon signed-rank test on the 20 per-PR scores and reported paired effect size d_z, defined as the mean paired difference divided by its standard deviation. The stated significance threshold was $p < 0.05$. No multiple-comparison correction was applied because there was one primary comparison.

5. Results

5.1 Overall comparison

Table 2. Overall comparison of the plain and structured prompts

Measure	Plain prompt	Structured prompt
Mean rating	3.80	4.05
Standard deviation	0.55	0.48
Median rating	4.0	4.0
Mean response time	2.9 s	3.4 s
Mean summary length	112 words	124 words
Summaries over 150 words	0	1

5.2 Ratings

The structured prompt's mean score was 4.05, compared with 3.80 for the plain prompt, a difference of 0.25 on the five-point scale. The Wilcoxon signed-rank test gave $p = 0.053$, and the paired effect size was $d_z = 0.45$. The p-value was just above the 0.05 threshold, so the result did not meet that stated threshold for statistical significance. With only 20 PRs, it should not be treated as conclusive.

5.3 Per-PR wins and differences

The structured prompt won on 11 PRs, the plain prompt on five, and four were ties. The structured format may be most helpful for diffs spanning several files, while a plain summary may be sufficient for small, focused changes.

5.4 Inter-rater agreement

Quadratic weighted kappa was 0.58, which indicates moderate agreement. Most disagreements were one point apart. The author's average preference for the structured prompt was slightly larger than the second rater's.

5.5 Response time and length

Mean response times were 2.9 seconds for the plain prompt and 3.4 seconds for the structured prompt. Mean output lengths were 112 and 124 words, respectively. No plain summary exceeded the 150-word limit, while one structured summary did.

5.6 Qualitative observations

- Structured prompt—strength: the Reason section can make intent in the PR title or code comments easier to locate.
- Structured prompt—weakness: the Risks section may include generic warnings without evidence in the diff.
- Plain prompt—strength: free-form summaries can read smoothly and remain efficient for small PRs.
- Plain prompt—weakness: summaries may list changes file by file, omit rationale, or leave out secondary edits in larger diffs.
- Both prompts: summaries of the largest diffs may miss changes because a 150-word budget cannot cover every detail.

6. Discussion

6.1 Answer to the research question

The structured prompt had a modestly higher average rating. The difference did not reach the 0.05 significance threshold. The effect was limited, and the plain prompt remained competitive, particularly for short PRs.

6.2 Possible explanation

Labelled sections may encourage the model to state the rationale for a change and consider reviewer-facing risks—two elements that a plain summary may omit. The trade-offs are longer output, slightly slower responses, and occasional generic risk statements. A future experiment could vary one component at a time, such as requesting Changes and Reason without Risks.

6.3 Practical implications

- Use the structured format when the goal is to help reviewers triage PRs and identify areas that deserve attention.
- Tell the model to mention risks only when the diff provides a basis for them, and allow “No notable risks” when appropriate.

- Offer a plain or short mode for small, focused PRs where brevity matters more than a fixed three-part format.

7. Threats to Validity and Limitations

7.1 Internal validity

One rater is the author, who designed the prompts and may have expected the structured prompt to perform better. Blinding is imperfect because the labelled sections reveal the structured format. Randomizing API-call and display order can reduce some order effects but does not remove these biases.

7.2 External validity

The study covers 20 PRs from eight repositories, mainly Python and JavaScript libraries. It may not generalize to other languages, very large diffs, industrial codebases, or PRs combining many unrelated changes.

7.3 Construct validity

A five-point usefulness score is subjective and may not correspond directly to improvements in review speed or quality. The study does not measure time spent reviewing, review accuracy, or decisions, and it omits reference summaries and complementary automatic measures such as ROUGE.

7.4 Conclusion validity

Twenty paired observations and two raters provide limited statistical power. A p-value close to the chosen threshold could change with another sample.

7.5 Model and run effects

The setup uses one Gemini model at temperature 0 on a single date [TO DO: add run date]. Other models, versions, and settings may produce different results.

8. Conclusion and Future Work

This paper compares a plain PR-summary prompt with a structured prompt requiring Changes, Reason, and Risks. We used 20 merged PRs, a fixed Gemini model at temperature 0, a 150-word limit, and human ratings. The results show a small average-rating advantage for the structured prompt, alongside slightly longer and slower outputs. The practical suggestion is to consider structured summaries for review triage while preventing unsupported generic risk statements.

Future work could use a larger PR dataset and more raters, compare several models, include reference-based evaluation, and measure how quickly reviewers understand a PR under each prompt. Other prompt formats, including few-shot examples and role instructions, could also be investigated.

Data and Code Availability

The scripts, prompts, generated summaries, and ratings are available at [TO DO: add repository URL after uploading them].

References

[1] Z. Liu, X. Xia, C. Treude, D. Lo, and S. Li, "Automatic Generation of Pull Request Descriptions," in Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2019, pp. 176–188. https://doi.org/10.1109/ASE.2019.00026

[2] W. Sun et al., "Automatic Code Summarization via ChatGPT: How Far Are We?" arXiv:2305.12865, 2023.

[3] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," arXiv:2201.11903, 2022.

[4] Haritha Raj, "pr-summarizer," GitHub repository: https://github.com/haritha1199/pr-summarizer.